TAKE-HOME PROJECT · WALKTHROUGH

Elevator System Simulation

A discrete-time **Destination Dispatch** simulation — pluggable schedulers, guaranteed constraints, and a reproducible fairness-vs-efficiency study.

`Python 3.13` `discrete-time engine` `SCAN · capacity · express` `3 pluggable schedulers`

Kaushik Ghosh
Design & implementation

Deterministic · seeded · reproducible

```
python -m elevator_sim --help
```

THE PROBLEM & HOW I APPROACHED IT

Three requirements → three design decisions

NOT THIS

A conventional ~~up/down hall-button~~ elevator where you press a direction and hope.

IT'S A

Destination Dispatch system — you declare your destination up front, and the controller assigns you a car **immediately**.

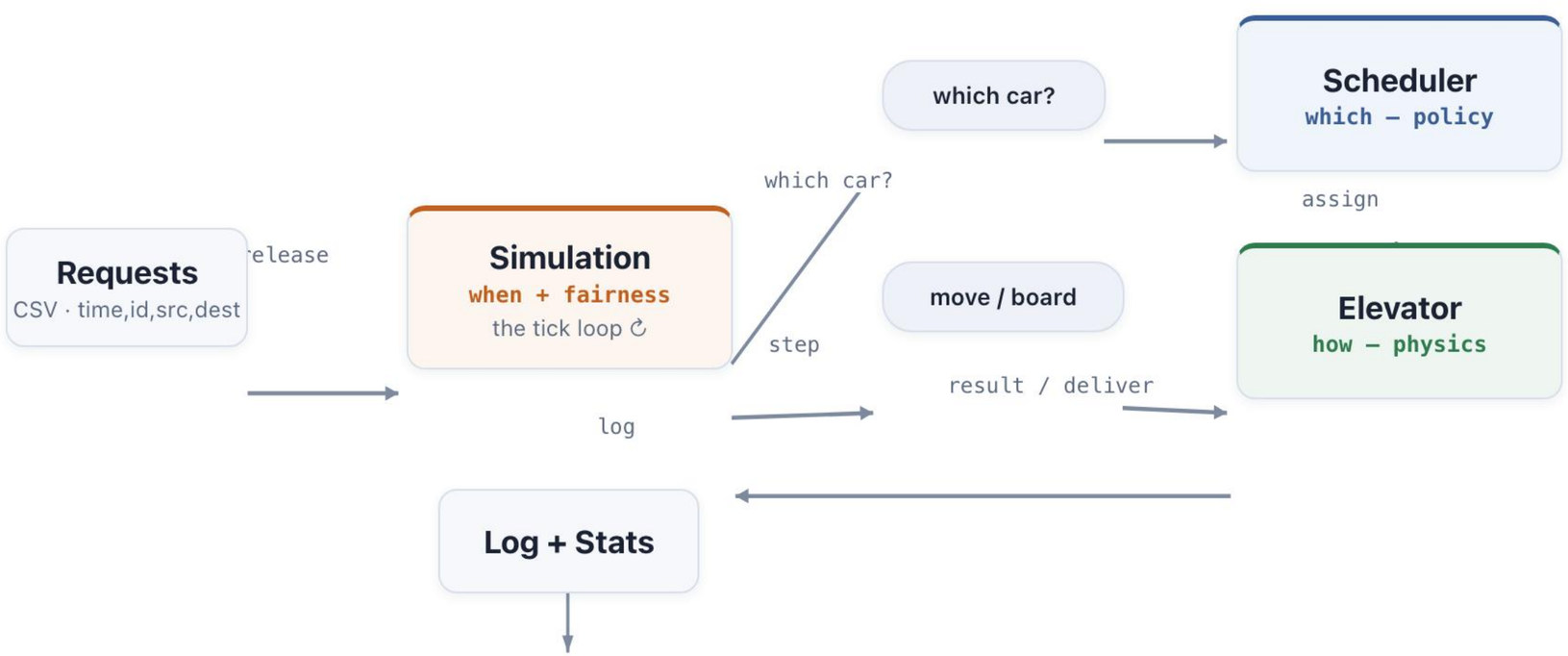
	THE ASK	HOW MY SOLUTION MEETS IT
1	**Serve everyone** No one waits forever.	**SCAN sweep guarantees every stop is served, and a car is always offered when capacity exists.** `Simulation · oldest-first dispatch`
2	**Minimize total time** Wait + travel.	**Nearest-car cost: assign to the car that reaches you soonest — not nearest by raw distance.** `Scheduler · eta + congestion`
3	**Honor constraints** Capacity, direction, floors.	**Elevator owns the physics — 1 floor/tick, capacity, direction — so all schedulers inherit them.** `Elevator · SCAN + capacity + express`

One idea threads all three: **separate *which* car (policy) from *how* it moves (physics)** — constraints live in one place, strategies stay swappable.

ARCHITECTURE · END-TO-END FLOW

High-level flow — requests in, stats out, one deterministic tick loop

`Simulation` orchestrates the loop; for each pending passenger it asks the `Scheduler` *which* car, then the `Elevator` executes *how* it moves.



SCHEDULER COST — HOW POLICY CHOOSES

SERVICE COST =

`eta_to(source)`
`+ stop_cost · commitment`

argmin over cars that can_accept · lower = better

Simulation decides **`when`** + fairness — clock, oldest-first dispatch, no-peek

Scheduler decides **`which`** car — efficiency / policy (the cost above)

Elevator decides **`how`** it moves — SCAN, capacity, express

Loop runs until `delivered == total` or a safety cap `T ≈ last_release + 8·F·C` — then raises, never hangs. No wall-clock, no randomness ⇒ **reproducible**.

SCHEDULING · GREEDY NEAREST-CAR

The scheduler decides which car — by minimizing a service cost

Destination Dispatch: origin + destination known at call time. Each passenger is assigned **greedily, oldest-first** to the lowest-cost car that can take them.

1 **Two-layer split.** Scheduler = *policy* (which car); Elevator = *physics* (how it moves). Every strategy inherits SCAN, capacity & express for free.

2 **ETA models the committed sweep, not raw distance.** A car heading away must finish its run before doubling back — so wrong-direction & busy cars are correctly expensive.

✓ **Liveness guarantee.** A car is returned whenever any has spare capacity; SCAN serves every stop. **No passenger waits forever** — and fairness lives in the simulation, not here.

! **Greedy & per-passenger.** Assigns one at a time as requests stream in — simple and starvation-free, but not jointly optimal within a tick (batching / min-cost-flow would tighten it).

SERVICE COST — LOWER IS BETTER

`eta_to(src) + stop_cost·commitment`

`eta_to(src)`	`commitment`
ticks to pickup along the SCAN run	onboard + assigned → spreads load

choose = `argmin` over cars where `can_accept(p)` · else retry next tick

— no aging term: it was identical across cars, a no-op in argmin, and removed.

THREE PLUGGABLE STRATEGIES (STRATEGY PATTERN)

`nearest_car` DEFAULT
Lowest service cost. Best avg latency.

`round_robin`
Cycle cars. Fair spread, ignores distance.

`zone_based`
Floor zones per car, nearest fallback.

Add a strategy: subclass `Scheduler` + `@register` — zero changes to `Simulation` or CLI.

DATA MODEL · PASSENGER

The passenger model — derive, don't store

State and every timing metric are **computed on read** from a handful of timestamps — the single source of truth — so they can never drift out of sync.

LIFECYCLE — STATE IS DERIVED FROM TIMESTAMPS + ASSIGNMENT, NEVER TRACKED BY HAND

Stored RAW FACTS

The only things actually written to the object.

`id` `source` `dest` `request_time` `pickup_time | None`
`dropoff_time | None` `assigned_elevator | None`

The three | None fields fill in as the passenger is served — that's the entire mutable surface.

Derived @property, READ-ONLY

Computed each access — nothing to keep in sync.

```
state ← which timestamps are set
wait_time = pickup_time − request_time
travel_time = dropoff_time − pickup_time
total_time = dropoff_time − request_time
direction = between(source, dest)
```

No drift
State & timing are pure functions of the timestamps — they can never contradict the underlying facts.

No invalid states
Can't be DELIVERED without a dropoff_time. The lifecycle is implied by the data, not asserted.

No sync bugs
There's no "update the flag *and* the timestamp" code path that can fall out of step.

Read-only by design: you set a timestamp; the state follows. `p.state` is a `@property`, not a field you can fake.

RESULTS · FAIRNESS VS. EFFICIENCY

No universal winner — the best scheduler depends on the traffic

All three deliver **100% of passengers** (completeness is guaranteed). The trade-off is in **average & tail latency**, not whether anyone is served. Seeded, reproducible.

MIXED TRAFFIC — 200 PASSENGERS · 60 FLOORS · 4 CARS · CAP 8 · SEED 42

Scheduler	avg total	max wait	p95 total	distance	delivered
nearest_car	134.9	241	236.0	1990	200/200
round_robin	146.8	265	256.1	2227	200/200
zone_based	144.9	277	253.2	2123	200/200

On realistic mixed traffic `nearest_car` wins: ~8% lower avg total *and* the lowest worst-case wait. Lower is better on every column.

BEST BY TRAFFIC PATTERN (LOWEST AVG TOTAL)

Mixed traffic (200)	`nearest_car · 134.9`
Tall building (300)	`nearest_car · 131.4`
Up-peak rush (150)	`round_robin · 210.5`
Provided sample (15)	`round_robin · 73.1`
Down-peak rush (150)	`zone_based · 217.3`

Why there's no single winner

nearest_car's distance/direction awareness only pays off when traffic is **spread out**. When everyone clusters at the lobby (up-peak), "nearest" barely differentiates and the system is throughput-bound — so blind round-robin spreading ties or wins. Contiguous zones happen to match down-peak's "upper floors → lobby" flow. **The winner is a function of the traffic shape.**

Fairness, measured honestly

Fairness here = **max wait** (the starvation check) + p95 (tail). A dispersion metric — `Gini` / stddev of wait — would let us plot the true Pareto frontier; that's a known next step, not a claim already made.

Completeness is not the trade-off

Every scheduler delivers everyone — guaranteed by the simulation, independent of strategy.

Efficiency edge grows with scale

nearest_car's lead widens from ~8% (mixed) to ~20% (tall building) as bundling pays off.

Pick the strategy for the traffic

Pluggable by design — the right default is nearest_car, but the bank can swap per building.

Reproduce: `python bench.py` · full sweep `python scripts/generate_results.py` → `results/*.{csv,json,png}`

THEORETICAL ANALYSIS · PER-TICK TIME COMPLEXITY

Dispatch is O(P · C · k) per tick — realistic load has orders of magnitude of headroom

P = passengers **concurrently pending** · C = cars · k = capacity · F = floors. Note: **F does not appear** — SCAN math is O(k) arithmetic over target *sets*, never a floor scan.

SORT PENDING
`O(P log P)`

+

DISPATCH: EACH P × EACH C × CHOOSE
`O(P · C · k)`

+

STEP ALL CARS (ALIGHT/BOARD)
`O(C · k)`

=

≈ O(P · C · k)

Realistic building

5,000 occupants

Morning up-peak: ~12% arrive in busy 5 min → ~2 calls/s. Pending queue stays small.

P (pending)	C (cars)	k (capacity)
~100	16	16

```
per-tick = P · C · k
= 100 · 16 · 16
= 25,600 ops / tick
```

≈ microseconds / tick

full-day sim (≈86,400 ticks): seconds total

✓ Comfortably within envelope. The design is built for exactly this scale.

Hypothetical stress

adversarial

Saturated: 10k passengers, far more than 50·16 = 800 slots. Pending queue stays huge and persistent.

P (pending)	C (cars)	k (capacity)
~10,000	50	16

```
per-tick = P · C · k
= 10,000 · 50 · 16
= 8,000,000 ops / tick
```

≈ 300× the realistic load

+ O(P log P) re-sort of a blocked queue every tick

⚠ Bottleneck = futile re-dispatch of a large blocked queue. Fix: heap-ordered pending + early-exit when no capacity.

Whole run ≈ O(T · P · C · k), where ticks `T ≈ last_release + 8·F·C`. Realistic P is **concurrent**, not the 5,000 daily total — the distinction dissolves the concern.

ROADMAP · TOWARD REAL DESTINATION-DISPATCH INTELLIGENCE

Potential improvements — from greedy per-passenger to joint optimization

Ordered by impact, tagged by effort. The first two close the biggest correctness gap; the rest add fidelity.

Today: greedy + permanent — assigns `one passenger at a time` as requests stream in, never re-homes. Simple and starvation-free, but not jointly optimal within a tick.

1 BIGGEST WIN

Batch / joint assignment

`choose_batch()`

Solve all **same-tick arrivals together** instead of one-at-a-time. Fixes greedy's globally-bad picks at **morning up-peak** — the classic DD scenario.

Effort: medium · min-cost flow

2 ~20–30%

Destination grouping

`co-directional bundles`

Bundle riders heading to the **same floors** into one car + skip-stop routing — the real throughput gain that defines destination dispatch.

Effort: high · set-partition

3 TAIL ↓

Dynamic reassignment

`re-optimize on free`

Re-home not-yet-boarded calls as cars free up. Cuts the **wait tail** (today's max wait 241); best suited to hall-call systems.

Effort: medium

4 FAIRNESS

Richer cost model

`+ per-car aging`

Account for pickups inserted after assignment; add a **per-car aging** term (survives argmin) and sweep **stop_cost** for a fairness/efficiency dial.

Effort: low · param sweep

5 FIDELITY

Physical fidelity

`real-world dynamics`

Door **dwell time**, acceleration/deceleration, multi-car shafts, and **sky-lobby** modeling for tall buildings.

Effort: high · modeling

Correctness / throughput · Quality of service · Realism — highest impact → polish ▶

①②③ all want the `choose_batch()` contract; `choose()` stays as the backward-compatible default — no rewrite of `Simulation`.